

1. Program Execution

Definition:

A Python program is executed from top to bottom. Definitions (classes, functions) are loaded first, and the executable code runs immediately when Python reaches it.

Example:

```
print("Hello")  
print("World")
```

Output:

```
Hello  
World
```

Best Practice:

In production code, executable logic should be inside:

```
if __name__ == "__main__":  
    main()
```

This avoids accidental execution when the file is imported.

Virtual Environment in Python

A virtual environment in Python is an isolated space where you can work on your Python projects independently from the system-installed Python. This allows you to manage dependencies and libraries specific to each project without affecting other projects or the system Python.

Why Use a Virtual Environment?

Virtual environments are essential for managing dependencies and avoiding conflicts between different projects. For instance, if you have two projects that require different versions of the same library, a virtual environment allows you to keep these dependencies separate. This isolation ensures that changes in one project do not affect another.

Creating a Virtual Environment

To create a virtual environment, you can use the `venv` module, which is included in Python 3.4 and above. Here's how you can create a virtual environment:

```
python -m venv my_env
```

This command creates a directory named `my_env` containing all the necessary executables to use the packages that a Python project would need.

Activating a Virtual Environment

After creating a virtual environment, you need to activate it. Activation ensures that the virtual environment's Python interpreter and libraries are used instead of the system-wide ones.

On Windows:

```
my_env\Scripts\activate
```

On Linux and macOS:

```
source my_env/bin/activate
```

Once activated, the name of your virtual environment will appear on the left side of the terminal prompt, indicating that it is active.

Installing Dependencies

With the virtual environment activated, you can install dependencies using `pip`. For example, to install Django 1.9, you would run:

```
pip install Django==1.9
```

This installs Django 1.9 in the virtual environment, keeping it isolated from the system Python.

Deactivating a Virtual Environment

When you are done working in the virtual environment, you can deactivate it by running:

```
deactivate
```

This command returns you to the system's default Python installation.

Deleting a Virtual Environment

To delete a virtual environment, simply deactivate it and remove the directory:

```
deactivate  
rm -r my_env
```

This removes the virtual environment and all its contents.

Conclusion

Virtual environments are a powerful tool for managing project-specific dependencies and avoiding conflicts. They are easy to create, activate, deactivate, and delete, making them an essential part of any Python developer's toolkit.

2. Variables & Naming Conventions

What you used:

```
title = input("Task Title : ")  
priority = input("Please enter valid priority")
```

Definition:

A variable is a named reference to data stored in memory.

Rules:

- Must start with a letter or `_`
- Cannot start with a number
- Case-sensitive

Beginner Example:

```
age = 30  
name = "Somnath"
```

Real-world Example:

```
task_due_date = "2026-02-10"
```

3. Data Types

The following are standard or built-in data types in Python:

- **Numeric:** `int`, `float`, `complex`
- **Sequence Type:** `string`, `list`, `tuple`
- **Mapping Type:** `dict`
- **Boolean:** `bool`
- **Set Type:** `set`, `frozenset`
- **Binary Types:** `bytes`, `bytearray`, `memoryview`

Examples:

```
x = 50 # int
x = 60.5 # float
x = "Hello World" # string
x = ["geeks", "for", "geeks"] # list
x = ("geeks", "for", "geeks") # tuple
```

String Operations:

```
s = 'Welcome to the Geeks World'
print(s)
# Check data type
print(type(s))
# Access string with index
print(s[1])
print(s[2])
print(s[-1])
```

4. Input & Output

What you used:

```
input("Task Title : ")
print("Task Manager Started...")
```

Definition:

- `input()` reads user input as a string.
- `print()` displays output.

Beginner Example:

```
name = input("Enter name: ")
print(name)
```

Important Note:

`input()` always returns a string. Conversion is required for other data types:

```
index = int(input("Enter number: "))
```

5. Conditional Statements

Conditional statements in Python allow you to execute specific blocks of code based on certain conditions. They are essential for decision-making in programs and are widely used in Class 11 IP (Informatics Practices) assignments.

Example Program: Student Grade Evaluation

Program to evaluate student grade based on marks:

```
# Program to evaluate student grade based on marks
marks = int(input("Enter your marks: "))

if marks >= 90:
    grade = "A+"
elif marks >= 80:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 60:
    grade = "C"
elif marks >= 50:
    grade = "D"
```

```
else:
    grade = "Fail"

print(f"Your Grade is: {grade}")
```

Explanation:

- `if` checks the first condition (marks ≥ 90).
- `elif` allows checking multiple conditions without nesting.
- `else` executes when none of the above conditions are true. This structure ensures that only one block runs, making the program efficient.

Example Program: Nested Conditional Statement

Program to check voting eligibility with membership benefits:

```
# Program to check voting eligibility with membership benefits
age = int(input("Enter your age: "))
is_member = input("Are you a registered member? (yes/no): ").lower()

if age >= 18:
    if is_member == "yes":
        print("Eligible to vote with priority access.")
    else:
        print("Eligible to vote.")
else:
    print("Not eligible to vote.")
```

Explanation:

- A nested `if` is used to check an additional condition inside the main condition.
- This is useful when one decision depends on another.

Tips for Writing Conditional Programs

- Use `elif` instead of multiple `if` statements to avoid unnecessary checks.
- For simple one-line conditions, use ternary operators:

```
result = "Pass" if marks >= 40 else "Fail"
```

This approach helps in writing more efficient and readable code.

6. Loops

In Python, loops are used to execute a block of code repeatedly until a certain condition is met. There are three main types of loops in Python: `for` loop, `while` loop, and nested loops.

For Loop

The `for` loop is used for iterating over a sequence (such as a list, tuple, string, or range). It executes a block of code for each item in the sequence.

Syntax:

```
for item in sequence:
    # code block
```

Example:

```
for i in range(5):
    print(i)
```

Output:

```
0
1
2
3
4
```

- The `range(5)` function generates a sequence of numbers from 0 to 4.

While Loop

The `while` loop executes a block of code as long as a specified condition is true.

Syntax:

```
while condition:
    # code block
```

Example:

```
count = 0
while count < 3:
    print("hello, world")
    count += 1
```

Output:

```
hello, world
hello, world
hello, world
```

- The loop continues to execute until the condition `count < 3` becomes false.

Nested Loops

Nested loops are loops within loops. You can use any type of loop inside another loop.

Syntax:

```
for i in range(3):
    for j in range(2):
        print(i, j)
```

Example:

```
for i in range(1, 4):
    for j in range(i):
        print(i, end=' ')
    print()
```

Output:

```
1
2 2
3 3 3
```

- This example demonstrates a nested `for` loop to create a triangular pattern.

Loop Control Statements

Python provides loop control statements to change the execution flow:

- **break:** Terminates the loop.

- **continue:** Skips the current iteration and continues with the next.
- **pass:** Does nothing and is used as a placeholder.

Example of break:

```
for letter in 'python':  
    if letter == 'h':  
        break  
    print('current letter:', letter)
```

Output:

```
current letter: p  
current letter: y  
current letter: t
```

- The loop terminates when it encounters the letter 'h'.

Summary:

Python provides versatile looping constructs to handle various iteration requirements efficiently. Understanding these loops and their control statements is essential for effective programming in Python.

7. Functions

A function in Python is a reusable block of code that performs a specific task. You define it using the `def` keyword, optionally accept parameters, and can return a value.

Example:

```
def greet_user(name):  
    """Display a greeting message."""  
    print(f"Hello, {name}!")  
  
greet_user("Alice")
```

- This defines `greet_user` with one parameter `name` and calls it with "Alice".

Basic Syntax

```
def function_name(parameters):  
    """Optional docstring describing the function."""
```

```
# Function body
return value # optional
```

- `def` starts the function definition.
- The function name should be descriptive.
- Parameters are optional.
- `return` sends data back to the caller; if omitted, returns `None`.

Positional and Keyword Arguments

```
def calculate_area(length, width=2):
    return length * width

print(calculate_area(5)) # Uses default width
print(calculate_area(width=3, length=5)) # Keyword arguments
```

- Default arguments simplify calls, while keyword arguments allow flexibility in order.

Variable-Length Arguments

```
def show_details(*args, **kwargs):
    print("Args:", args)
    print("Kwargs:", kwargs)

show_details(1, 2, 3, name="John", age=30)
```

- `*args` handles extra positional arguments.
- `**kwargs` handles extra keyword arguments.

Returning Values

```
def is_even(num):
    return num % 2 == 0

if is_even(4):
    print("Even")
```

- Functions can return any type — numbers, strings, lists, or even other functions.

Nested and Anonymous Functions

```
def outer():  
    def inner():  
        print("Inner function")  
    inner()
```

```
square = lambda x: x * x
```

- Nested functions help encapsulate logic.
- `lambda` creates small anonymous functions.

Key Considerations

- Always use meaningful names.
- Keep functions focused on a single task.
- Use docstrings for clarity.
- Avoid mutable default arguments to prevent unexpected behavior.

By mastering function definitions, you write cleaner, reusable, and more maintainable Python code.

8. Python Classes and Objects

In Python, classes are blueprints for creating objects, and objects are instances of these classes. Classes encapsulate data (attributes) and behavior (methods), making it easier to manage and reuse code. Below are examples to illustrate the concepts of classes and objects.

Defining a Class and Creating Objects

A class is defined using the `class` keyword. Objects are created by instantiating the class.

```
# Define a class  
class Dog:  
    sound = "bark" # Class attribute  
  
# Create an object  
dog1 = Dog()  
  
# Access the class attribute  
print(dog1.sound) # Output: bark
```

Here, `Dog` is the class, and `dog1` is an object of the class. The `sound` attribute is shared across all instances of the class.

Using the `__init__` Method

The `__init__` method is a constructor that initializes instance attributes when an object is created.

```
class Dog:
    species = "Canine"  # Class attribute

    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age  # Instance attribute

# Create an object
dog1 = Dog("Buddy", 3)

# Access attributes
print(dog1.name)  # Output: Buddy
print(dog1.species)  # Output: Canine
```

The `__init__` method allows each object to have unique values for its instance attributes.

Adding Methods to a Class

Methods are functions defined inside a class that operate on its attributes.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name} is barking!")

# Create an object and call a method
dog1 = Dog("Buddy", 3)
dog1.bark()  # Output: Buddy is barking!
```

The `self` parameter refers to the current instance of the class, allowing access to its attributes and methods.

Custom String Representation with `__str__`

The `__str__` method provides a readable string representation of an object.

```

class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name} is {self.age} years old."

# Create objects
dog1 = Dog("Buddy", 3)
dog2 = Dog("Charlie", 5)

# Print objects
print(dog1)  # Output: Buddy is 3 years old
print(dog2)  # Output: Charlie is 5 years old

```

Without `__str__`, printing an object would display a less informative memory address.

Class vs. Instance Variables

Class variables are shared across all instances, while instance variables are unique to each object.

```

class Dog:
    species = "Canine"  # Class variable

    def __init__(self, name, age):
        self.name = name  # Instance variable
        self.age = age  # Instance variable

# Create objects
dog1 = Dog("Buddy", 3)
dog2 = Dog("Charlie", 5)

# Access and modify variables
print(dog1.species)  # Output: Canine
dog1.name = "Max"
print(dog1.name)  # Output: Max
Dog.species = "Feline"
print(dog2.species)  # Output: Feline

```

Class variables are accessed using the class name, while instance variables are accessed through the object.

9. Lists, Dictionaries, Tuples, and Sets

List

- **Definition:** A list is an ordered, mutable collection of elements enclosed in square brackets `[]`. It allows duplicate elements and can store heterogeneous data types.

- **Example:**

```
fruits = ['apple', 'banana', 'orange']
fruits.append('grape') # Add an element
print(fruits)
# Output: ['apple', 'banana', 'orange', 'grape']
```

- **Use Cases:** Ideal for dynamic arrays, data manipulation, and implementing stacks or queues.

Tuple

- **Definition:** A tuple is an ordered, immutable collection of elements enclosed in parentheses `()`. Once created, its elements cannot be modified. Tuples are often used for fixed collections of related data.

- **Example:**

```
person = ('John', 25, 'New York')
print(person[0]) # Access the first element
# Output: John
```

- **Use Cases:** Memory-efficient and can be used as dictionary keys due to their immutability.

Set

- **Definition:** A set is an unordered, mutable collection of unique elements enclosed in curly braces `{}`. It does not allow duplicate values and is useful for mathematical operations like union, intersection, and difference.

- **Example:**

```
numbers = {1, 2, 3, 4, 4, 5}
print(numbers)
# Output: {1, 2, 3, 4, 5}
```

- **Use Cases:** Efficient for membership testing and removing duplicates from a collection.

Dictionary

- **Definition:** A dictionary is an unordered (ordered since Python 3.7), mutable collection of key-value pairs enclosed in curly braces `{}`. Keys must be unique and are used to access their corresponding values.
- **Example:**

```
student = {'name': 'Alice', 'age': 20, 'major': 'Computer Science'}  
print(student['name']) # Access value by key  
# Output: Alice
```

- **Use Cases:** Ideal for fast lookups, representing structured data, and counting frequencies.

Key Differences

- **List:** Ordered, mutable, allows duplicates.
- **Tuple:** Ordered, immutable, allows duplicates.
- **Set:** Unordered, mutable, does not allow duplicates.
- **Dictionary:** Unordered (ordered since Python 3.7), mutable, keys must be unique.

Examples of Usage

- **List Example:**

```
numbers = [1, 2, 3]
```

- **Dictionary Example:**

```
person = {"name": "Somnath", "age": 30}
```

- **Real-world Example:**

- Grouping tasks by priority:

```
self.taskList = []  
prTasks = {}
```

10. Handling Errors in Python

Errors are an inevitable part of programming, and handling them effectively is crucial for creating robust and reliable applications. Python provides several mechanisms to handle errors gracefully,

ensuring that the program can recover from unexpected situations and continue executing.

Using `try` and `except`

The `try` block allows you to test a block of code for errors, while the `except` block lets you handle the error if it occurs. This prevents the program from crashing and allows you to manage the error appropriately.

```
try:
    a = int(input("Enter a number: "))
    b = int(input("Enter another number: "))
    result = a / b
    print(result)
except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")
except ValueError:
    print("Error: Invalid input. Please enter a valid number.")
```

In this example, if the user enters `0` for `b`, a `ZeroDivisionError` is raised, and the corresponding `except` block handles it.

Using `else` and `finally`

The `else` block can be used to execute code if no errors are raised in the `try` block. The `finally` block is always executed, regardless of whether an error occurred or not. This is useful for cleaning up resources or performing actions that must occur regardless of the outcome.

```
try:
    a = int(input("Enter a number: "))
    b = int(input("Enter another number: "))
    result = a / b
except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")
except ValueError:
    print("Error: Invalid input. Please enter a valid number.")
else:
    print(f"The result is {result}")
finally:
    print("Execution completed.")
```


In this example, the `else` block executes only if no exceptions are raised, and the `finally` block executes regardless of the outcome.

Raising Exceptions

You can also raise exceptions intentionally using the `raise` statement. This is useful for creating custom error messages or handling specific conditions in your code.

```
def check_positive(number):  
    if number < 0:  
        raise ValueError("The number must be positive.")  
    return number  
  
try:  
    num = check_positive(-5)  
except ValueError as e:  
    print(e)
```

In this example, a `ValueError` is raised if the number is negative, and the `except` block handles it.

Summary

Handling errors in Python involves using `try`, `except`, `else`, and `finally` blocks to manage exceptions and ensure the program can recover from unexpected situations. By anticipating potential errors and handling them appropriately, you can create more robust and reliable applications.

11. PYTHON FILE HANDLING – COMPLETE GUIDE (CSV & JSON – Beginner to Advanced)

1 What is File Handling?

What it is

File handling involves:

- Creating files

- Reading data from files
- Writing data to files
- Updating existing files

Python allows you to persist data even after the program stops.

Why file handling is needed

- **Without files:**
 - All data is lost when the program exits.
- **With files:**
 - Data is saved permanently.
 - Programs become real applications.

Example:

- In your Task Manager, tasks are stored in `TaskManager.json`.
- When the app restarts, tasks are restored.

2 File Modes (VERY IMPORTANT)

Mode	Meaning
"r"	Read
"w"	Write (overwrites)
"a"	Append
"x"	Create new file
"rb"	Read binary
"wb"	Write binary

3 Basic File Handling Syntax

Without `with` (NOT recommended)

```
file = open("data.txt", "r")
content = file.read()
file.close()
```

With with (BEST PRACTICE)

```
with open("data.txt", "r") as file:
    content = file.read()
```

- **Benefits:**
 - Automatically closes the file.
 - Prevents memory leaks.
 - Cleaner code.

JSON FILE HANDLING (DEEP DIVE)

What is JSON?

What it is

JSON (JavaScript Object Notation) is a text format used to store structured data.

Example:

```
{
  "title": "Buy groceries",
  "priority": "High",
  "status": "Pending"
}
```

Why JSON is used

- Human-readable
- Language-independent
- Perfect for configs & storage

JSON Data Types Mapping

JSON	Python
Object	dict
Array	list
String	str

JSON	Python
Number	int / float
Boolean	bool
null	None

6 JSON Module in Python

```
import json
```

Function	Purpose
<code>json.load()</code>	Read JSON from file
<code>json.loads()</code>	Read JSON from string
<code>json.dump()</code>	Write JSON to file
<code>json.dumps()</code>	Convert to JSON string

7 Reading JSON File (Your Code Explained)

Your code:

```
with open("./TaskManager.json", "r") as file:
    data = json.load(file)
```

Step-by-step explanation:

- `open()` opens the file.
- `"r"` → read mode.
- `json.load(file)` converts JSON to a Python list or dict.
- Stored in `data`.

After this:

```
data == list of dictionaries
```

Example JSON file content:

```
[
  {
    "title": "Pay bills",
    "description": "Electricity bill",
    "dueDate": "2024-02-10",
    "priority": "High",
    "status": "Pending"
  }
]
```

8 Writing JSON File (Your Code Explained)

Your code:

```
json.dump(data, file, indent=4, default=str)
```

What each part means:

- `data` → Python list or dict.
- `file` → opened file object.
- `indent=4` → pretty formatting.
- `default=str` → converts datetime to string.

Why `default=str` is needed:

- JSON cannot store datetime objects.
- **Without it:**

```
TypeError: Object of type datetime is not JSON serializable
```

9 Writing JSON – Best Practice Version

```
def save_tasks(self):
    with open("TaskManager.json", "w") as file:
        json.dump(
            [task.to_dict() for task in self.taskList],
            file,
            indent=4
        )
```

COMMON JSON MISTAKES

- Forgetting file exists
- Writing datetime directly
- Overwriting data accidentally
- Using bare `except`

Best Practices:

- Always validate.
- Use `try/except`.
- Convert custom objects.

JSON with Custom Objects (VERY IMPORTANT)

Python cannot save objects directly.

Incorrect approach:

```
json.dump(self.taskList, file)
```

Correct approach:

- Convert object to dict.

Example:

```
def to_dict(self):
    return {
        "title": self.title,
        "description": self.description,
        "dueDate": self.due_date.strftime("%Y-%m-%d"),
        "priority": self.priority,
        "status": self.status
    }
```



CSV FILE HANDLING (DEEP DIVE)

What is CSV?

What it is

CSV (Comma Separated Values) stores tabular data.

Example:

```
title,priority,status  
Pay bills,High,Pending
```

When to use CSV

- Excel compatibility
- Reporting
- Data analysis

1 2 CSV Module

```
import csv
```

1 3 Writing CSV File

```
with open("tasks.csv", "w", newline="") as file:  
    writer = csv.writer(file)  
    writer.writerow(["Title", "Priority", "Status"])  
    writer.writerow(["Pay bills", "High", "Pending"])
```

Why `newline=""` ?

- Prevents blank lines on Windows.

1 4 Reading CSV File

```
with open("tasks.csv", "r") as file:  
    reader = csv.reader(file)  
    for row in reader:  
        print(row)
```

1 5 CSV with DictReader / DictWriter (BEST)

Writing:

```
with open("tasks.csv", "w", newline="") as file:
    writer = csv.DictWriter(
        file,
        fieldnames=["title", "priority", "status"]
    )
    writer.writeheader()
    writer.writerow({
        "title": "Pay bills",
        "priority": "High",
        "status": "Pending"
    })
```

Reading:

```
with open("tasks.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["title"])
```



CSV vs JSON (IMPORTANT COMPARISON)

Feature	CSV	JSON
Structure	Flat	Nested
Human-readable	Medium	High
Datatypes	No	Yes
Best for	Excel	Applications

13 Working with Dates and Times in Python

Python provides a powerful module called `datetime` to handle dates and times. This module includes several classes to represent and manipulate dates and times, as well as to format and parse them in various formats.

Creating Date and Time Instances

The `datetime` module includes three main classes: `date`, `time`, and `datetime`. These classes can be instantiated by passing keyword arguments for each attribute, such as `year`, `month`, `day`, `hour`, `minute`, and `second`.

Example: Getting Current Date and Time

```
import datetime

# Get the current date and time
now = datetime.datetime.now()
print(now)
```

Output:

```
2022-12-27 08:26:49.219717
```

In this example, the `now()` method creates a `datetime` object containing the current local date and time.

Example: Getting Current Date

```
import datetime

# Get the current date
current_date = datetime.date.today()
print(current_date)
```

Output:

```
2022-12-27
```

The `today()` method defined in the `date` class returns a `date` object containing the current local date.

Manipulating Dates and Times

The `datetime` module allows you to perform arithmetic operations on dates and times using the `timedelta` class. This class represents the difference between two dates or times.

Example: Difference Between Two Dates

```
from datetime import datetime, timedelta

# Define two dates
date1 = datetime(2022, 12, 25)
date2 = datetime(2022, 12, 20)

# Calculate the difference
difference = date1 - date2
print(difference)
```

Output:

```
5 days, 0:00:00
```

In this example, the difference between `date1` and `date2` is calculated using the subtraction operator, resulting in a `timedelta` object.

Formatting and Parsing Dates and Times

The `datetime` module provides methods to format and parse dates and times using the `strftime()` and `strptime()` methods.

Example: Formatting Date and Time

```
from datetime import datetime

# Get the current date and time
now = datetime.now()

# Format the date and time
formatted_date = now.strftime("%Y-%m-%d %H:%M:%S")
print(formatted_date)
```

Output:

```
2022-12-27 08:26:49
```

The `strftime()` method formats a `datetime` object into a string based on the specified format codes.

Example: Parsing Date and Time

```
from datetime import datetime

# Define a date string
date_string = "2022-12-25 08:26:49"

# Parse the date string
parsed_date = datetime.strptime(date_string, "%Y-%m-%d %H:%M:%S")
print(parsed_date)
```

Output:

2022-12-25 08:26:49

The `strptime()` method parses a string into a `datetime` object based on the specified format codes.

Handling Time Zones

To handle time zones, you can use the `pytz` module, which provides support for time zones.

Example: Working with Time Zones

```
from datetime import datetime
import pytz

# Get the current time in the local time zone
local_time = datetime.now()
print("Local time:", local_time)

# Get the current time in New York
ny_time = datetime.now(pytz.timezone('America/New_York'))
print("New York time:", ny_time)

# Get the current time in London
london_time = datetime.now(pytz.timezone('Europe/London'))
print("London time:", london_time)
```

Output:

Local time: 2022-12-27 09:40:19

New York time: 2022-12-27 04:40:19

London time: 2022-12-27 09:40:19

In this example, the `pytz` module is used to get the current time in different time zones.

By using the `datetime` module and the `pytz` module, you can effectively manage dates and times in your Python programs, including handling time zones and performing arithmetic operations on dates and times.

14. Python Operators Overview

In Python, operators are special symbols used to perform operations on variables and values. The values that the operators act upon are called operands. Python supports several categories of operators, each serving a specific purpose.

1. Arithmetic Operators

- **Purpose:** Perform mathematical operations.
- **Operators:**
 - `+` (Addition)
 - `-` (Subtraction)
 - `*` (Multiplication)
 - `/` (Division)
 - `//` (Floor Division)
 - `%` (Modulus)
 - `**` (Exponentiation)

```
a, b = 15, 4
print(a + b, a - b, a * b, a / b, a // b, a % b, a ** b)
```

2. Comparison Operators

- **Purpose:** Compare values and return `True` or `False`.
- **Operators:**
 - `==` (Equal to)
 - `!=` (Not equal to)
 - `>` (Greater than)
 - `<` (Less than)
 - `>=` (Greater than or equal to)
 - `<=` (Less than or equal to)

```
print(10 > 5, 10 == 5, 10 != 5)
```

3. Logical Operators

- **Purpose:** Combine conditional statements.
- **Operators:**
 - `and`
 - `or`
 - `not`

```
x, y = True, False  
print(x and y, x or y, not x)
```

4. Bitwise Operators

- **Purpose:** Operate on binary representations.
- **Operators:**
 - `&` (AND)
 - `|` (OR)
 - `~` (NOT)
 - `^` (XOR)
 - `<<` (Left Shift)
 - `>>` (Right Shift)

```
a, b = 10, 4  
print(a & b, a | b, ~a, a ^ b, a << 2, a >> 2)
```

5. Assignment Operators

- **Purpose:** Assign values with optional operation.
- **Operators:**
 - `=` (Simple assignment)
 - `+=` (Add and assign)
 - `-=` (Subtract and assign)
 - `*=` (Multiply and assign)
 - `/=` (Divide and assign)
 - `%=` (Modulus and assign)
 - `**=` (Exponentiate and assign)

- `//=` (Floor divide and assign)
- `<<=` (Left shift and assign)
- `>>=` (Right shift and assign)
- `&=` (Bitwise AND and assign)
- `|=` (Bitwise OR and assign)
- `^=` (Bitwise XOR and assign)

```
a = 5
a += 3 # Equivalent to a = a + 3
print(a)
```

6. Identity Operators

- **Purpose:** Compare memory locations.
- **Operators:**

- `is`
- `is not`

```
x = [1, 2]
y = x
print(x is y, x is not y)
```

7. Membership Operators

- **Purpose:** Check sequence membership.
- **Operators:**

- `in`
- `not in`

```
nums = [1, 2, 3]
print(2 in nums, 4 not in nums)
```

8. Ternary Operator

- **Purpose:** Conditional expression in one line.

```
a, b = 10, 20
min_val = a if a < b else b
print(min_val)
```

Operator Precedence and Associativity

- **Operator Precedence:** Determines the order in which operations are executed.
- **Associativity:** Resolves ties when precedence is equal.

Understanding operator precedence and associativity ensures the correct evaluation of complex expressions.

15 Using Regex in Python

Regular Expressions (regex) in Python are handled by the built-in `re` module, allowing you to search, match, split, and replace text based on patterns.

Example: Find all email addresses in a string

```
import re

text = "Contact us at support@example.com or sales@company.org"
pattern = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-z]{2,}'

emails = re.findall(pattern, text)
print(emails)  # ['support@example.com', 'sales@company.org']
```

Here, `re.findall()` returns all non-overlapping matches of the pattern in the string.

1. Searching and Matching

- `re.search(pattern, string)` : Finds the first match anywhere in the string.
- `re.match(pattern, string)` : Matches only at the start of the string.

```
m = re.search(r'\d+', "Order number: 12345")
print(m.group())  # 12345
```

2. Finding All Matches

- `re.findall(pattern, string)` : Returns a list of all matches.
- `re.finditer(pattern, string)` : Returns an iterator of match objects.

```
for match in re.finditer(r'\b\w{4}\b', "This test code runs fine"):
    print(match.group())    # This, test, code
```

3. Splitting Strings

Use `re.split()` to split by a regex delimiter:

```
parts = re.split(r'\W+', "Words, words; more words!")
print(parts)    # ['Words', 'words', 'more', 'words', '']
```

4. Substitution (Replacing)

Use `re.sub()` to replace matches:

```
result = re.sub(r'\d+', '#', "User123 has 456 points")
print(result)    # User# has # points
```

5. Compiling Patterns for Reuse

For repeated use, compile your regex:

```
date_pattern = re.compile(r'(\d{2})-(\d{2})-(\d{4})')
print(date_pattern.sub(r'\3/\2/\1', "26-08-2020"))    # 2020/08/26
```

Key Tips

- Use raw strings (`r"..."`) to avoid escaping backslashes unnecessarily.
- Common metacharacters: `.` `^` `$` `*` `+` `?` `{` `}` `[` `]` `\` `|` `(` `)` `.`
- Flags like `re.IGNORECASE`, `re.MULTILINE`, and `re.DOTALL` modify matching behavior.

This makes regex in Python a powerful tool for text validation, extraction, and transformation.